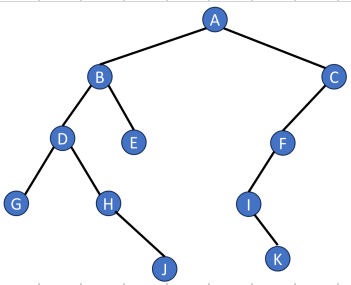
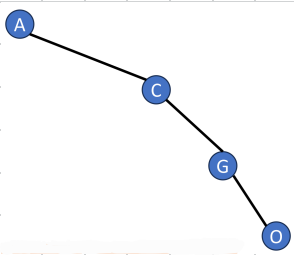
Tree

Binary Tree





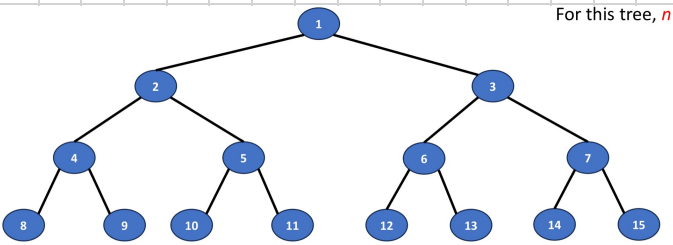
Let n be the number of nodes in a binary tree whose height is h

① $\Rightarrow h \le n \le 2^h - 1$

② $\log_2(n+1) \le h$

$\Downarrow$

the height h of a binary tree is at least $\log_2(n+1)$



For this tree, n = 15

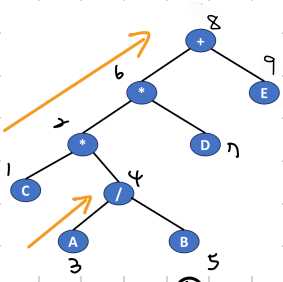
① Parent of node i is node $\text{floor}(i/2)$

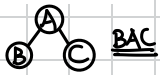
② Left child of node i is node $2i$

③ Right child of node i is node $2i+1$

可互相反推

Binary Tree Traversal



inorder 

BAC

Iterative

※ When A is pushed, the current is A

① Push root into stack

② Push left child into stack until reaching a null node.

③ Pop top of stack

④ Push right child into stack

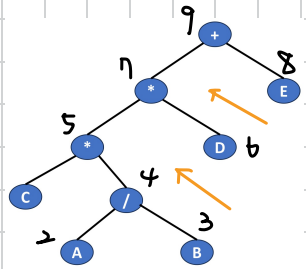
0	1	2	3	4	5	6	7
9	6	8					

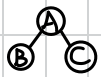
front	lastrear	rear
0	0	1
1	1	3
2	1	3
3	1	3

More

「successor」指的是下一個要 traverse 的節點

ex: If a node doesn't have any childs, based on "preorder traversal", the successor will be its parent's right child.



postorder  BCA

Iterative ①

① Set root as current

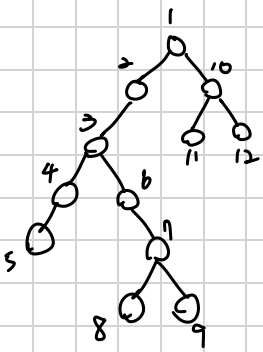
② Set left child as current and push it into stack until reaching a null node

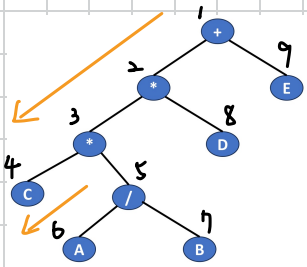
③ Pop top of stack and set top as current until right child is not a null node

④ Set right child as current and push it into stack

a.k.a 前序線索

Iterative ①





preorder  ABC

Iterative ①

① Set root as current and add it into queue

② Add left child into queue and push right child into stack and set left child as current until left child is a null node

③ Set top of stack as current and add it into queue and pop it from stack

Constructing binary tree from sequences

Preorder (R) ABCDEFGHI

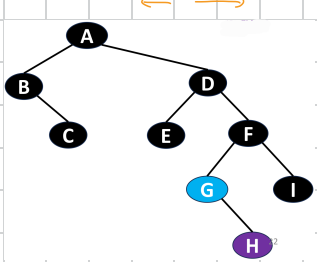
Inorder BCAEDGHFI

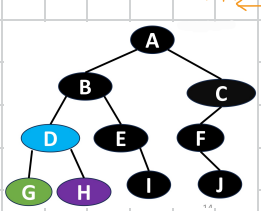
left = right

Postorder GHDIEBJFCA (R, leftmost leaf)

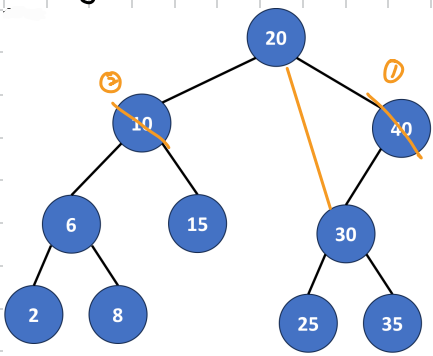
Inorder GDHBEIAFJC

left right





Binary Search Tree



the time complexity of search() is O(height)

insert() is O(height)

delete()

① delete a degree-1 node

② delete a degree-2 node:

- replace by the largest pair in its left subtree
- replace by the smallest pair in its right subtree

Heap

- A complete binary tree
- Min heap is a min tree
- Max heap is a max tree

Root has the smallest tree

※ the time complexity of insert() & delete() is O(height)

insert(key) — at end of the array — ∵ heap is a complete binary tree

Push the key into the tree array and it keeps comparing with its parent

delete(key)

① Remove the key from the tree (the key must be a root of a tree or subtree)

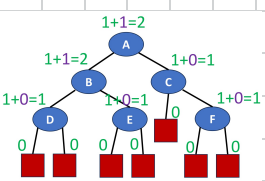
② Replace the position with the end of the tree array

③ Keeps comparing with its child

Leftist tree ※ 用處就是「高效合併」

HBLT (Height-based leftist tree)

- An extended binary tree
- For every internal node x, shortest(leftChild(x)) ≥ shortest(rightChild(x))



Property

- The rightmost path is a shortest root to external node path
- Number of internal nodes $n \geq 2^{shortest(root)} - 1$, $shortest(root) \leq \log_2(n+1)$

insert(x, T_a)
- Create a min leftist tree T_b containing only x
- Meld(T_a, T_b)

deleteMin(T_a)
- Meld(root→leftChild, root→rightChild)
- Delete the original root

Meld(T_a, T_b) ※ the time complexity of Meld() is $O(\log(n_A+n_B))$

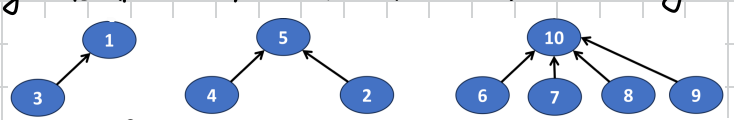
Phrase I (Top-down)
- Going down along the rightmost paths in T_a or T_b
- Compare their roots. For the tree with smaller,
  - If no rightChild, attach another tree as rightChild
  - If having rightChild, T = T→rightChild and repeat Phrase I

Phrase II (Bottom-up) → ※ 比這還下面的 subtrees 沒動過，一定符合 leftist tree
- Start from last modified root. Compare shortest(leftChild) and shortest(rightChild) to maintain the property of leftist tree
- T = T→parent

Initialization() ※ the time complexity of Initialization() is $n+2\cdot(1\cdot\frac{n}{2}+2\cdot\frac{n}{4}+\dots)=O(n)$
- With a queue of n single node min trees
- Repeatedly remove two min trees from the queue and meld them

$H_1, H_2, \dots, H_8 \longrightarrow H_{12}, H_{34}, H_{56}, H_{78} \longrightarrow H_{1234}, H_{5678} \longrightarrow H_{12345678}$

WBLT (Weight-based leftist tree) ※ count(): the number of internal nodes
- An extended binary tree
- For every internal node x, count(leftChild(x)) ≥ count(rightChild(x))



Disjoint sets

Use an integer array to store the parent of each elements in three disjoint sets

find(i)
- Trace parents until reaching root of the set. It help compare whether the roots of two elements are the same

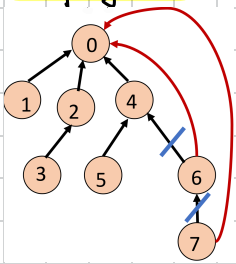
union(i, j)

parent[j] = i

Weight rule: make tree with fewer number of elements as subtree

Height rule: make tree with smaller height as subtree of the other tree

collapsing rule



- As find(7) is processing, we can make all nodes on find path point to tree root.

Graph

Undirected (u,v), (v,u)

u ——— v

Directed <u,v> <v,u>

u ⟶ v u ⟵ v

Simple path No repeating vertices

Cycle • simple path • 1st and the last vertices are the same

Strongly connected components

In a directed graph, every pair of vertices u and v has a directed path from u to v and also from v to u

Complete graph Having the maximum number of edges

※ Number of edges in a directed graph ≤ n(n-1)

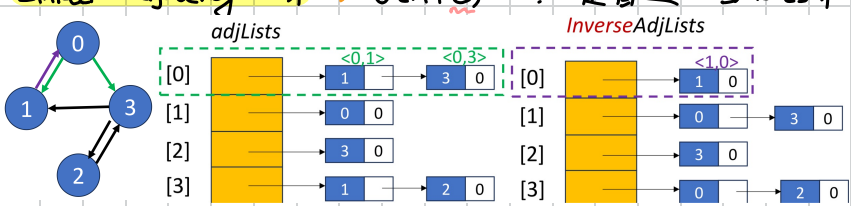
↳ n: number of vertices

Tree • A graph with no cycles • A connected graph
• n vertices with n-1 edges

Spanning Tree • A subgraph that includes all vertices of the original graph
• A tree

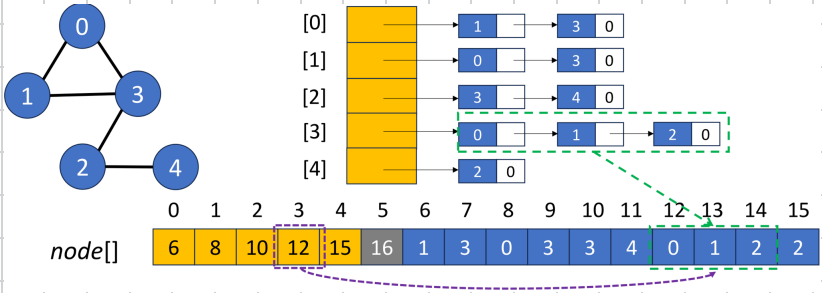
Linked Adjacency List ※ O(n+e) ∵ 一定看過 n個 nodes 和 e個 edges 且看過的不會再看

(the number of all edges)



※ Total number of chain nodes in unindirected graph: 2e
Total number of chain nodes in indirected graph: e

Array Adjacency List



Weighted Adjacency Matrix

※ $O(n^2)$

	0	1	2	3
0	0	1	0	2
1	1	0	0	3
2	0	0	0	5
3	2	3	5	0

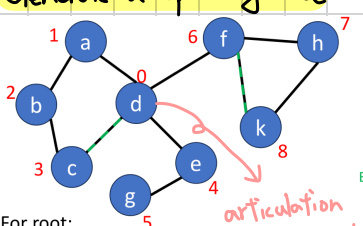
DFS (Depth-First)

```
for (each unreached vertex u adjacent from v)
    dfs(u);
```

BFS (Breadth-First)

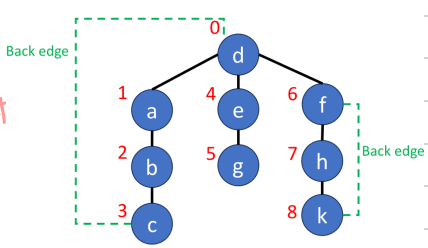
- Start at a vertex and put it into FIFO queue and visited vertices
- Set the front as current
- Put the adjacent unvisited vertices into FIFO queue and visited vertices

Generate a spanning tree



Generating a depth-first search spanning tree. Ex: dfs(d)

Nontree edges are back edges.

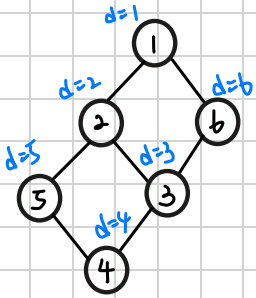


- For root:
  Number of children of root ≥ 2
  → Root is an articulation point.
- For a non-root vertex *v*:
  A child of vertex *v* cannot reach any ancestor of vertex *v* via other paths.
  → Vertex *v* is an articulation point.

28

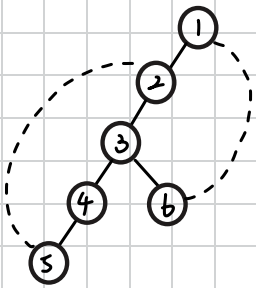
※ 遇到已訪問且不是 parent ⇒ 記為 "back edge"

Detect an articulation point — DFS



node	1	2	3	4	5	6
discovery time	1	2	3	4	5	6
Low d						

DFS spanning tree



Condition ① except root of the entire spanning tree

If `L[child] < d[parent]`

then parent is not an articulation point.

Reason: child 就算沒有 parent，也能知 parent 的 parent 相連

Condition ② for root

root 沒有 parent，所以不適用 condition ①

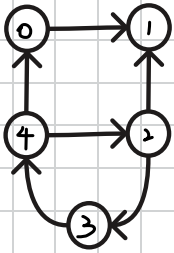
If root has only one child,

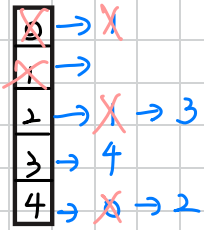
then root is not an articulation point.

Reason: 若 root 有 2 個以上的 child，代表這些 childs 間一定沒有相連

Detect an cycle

DFS ※ 我猜 undirected graph 也能用類似作法





ex: Set node 4 as a startpoint

0	1	2	3	4
F	F	F	F	T
T	F	F	F	T
T	T	F	F	T
T	F	F	F	T
F	F	F	F	T

Failed

Failed

F	F	T	F	T
T	F	T	F	T
F	T	T	F	T
F	F	T	F	T
F	F	T	T	T

若沒把 Failed 設回 0，這個時候會誤判

此時 4 也是 True

∴ A cycle is found.

0	4
1	0 →
2	4
3	2
4	3

Kahn

Proof of node of in-degree = 0 in DAG → 沒有環的 graph

out-degree = 0 也一樣成立

→ 假設每個點 in-degree ≥ 1，則此時任意點 v_0 往前找 $v_1, v_2, \cdots$

最後一定會發現這是 a cycle → 有 N 個 node (finite)

∴ DAG 一定存在 in-degree = 0 的點

Procedure:

過程中，其他 node 的 in-degree 也會減少

① Remove all nodes of in-degree = 0

② If there're any nodes remained, then this graph has a cycle

Minimum cost spanning tree

Kruskal's method

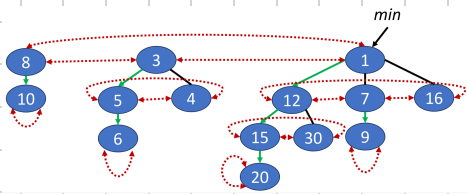
- Start with n-vertices 0-edge forest
- Choose the least cost edge that won't create a cycle until there are n-1 edges
  - the two vertices are not in the same set
  - If success, merge the two sets

Prim's method

- Start with the least cost vertice and push the two connected vertices into visited-vertices
- Choose the least cost vertice of all the adjacent vertices if it is not in visited vertices

Fibonacci heap

- Collection of min trees
- The min trees need NOT be binomial trees
- No search operation



insertion

- Add a new single-node min tree to the collection
- Update min-element pointer if necessary

meld

- Meld the top circular lists of two B-heaps into one circular list
- Update min-element pointer if necessary

delete

- Remove the accused node from its circular list
- Reinsert subtrees of the removed node
- Update min-element pointer
- Pairs of min trees having the same degree must be joined (The min tree whose root has a larger key becomes a subtree of the other one)

the value of degree can be represented by the index of an array

decreaseKey

- Decrease key
- If the node is not a root and new key < parent key,
  - Remove subtree rooted at the node from its doubly linked sibling list
  - Insert it into top-level list
  - Update min-pointer

ChildCut flag

True if node has lost a child; otherwise, False

Cascading cut

- When the node is cut out in a delete or decrease key operation, follow path from parent of the node to the root
- If ChildCut = true, then cut it from their sibling lists and inserted into top-level list until ChildCut = false. Then, set it to true

Binomial Tree B_k

節點數 $= 2^k$　　第 i 層節點數 $= C_i^k$

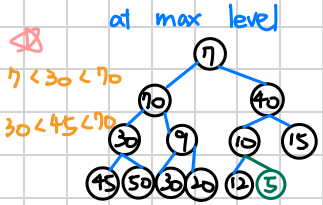
根的度數 $= k$　　根有 k 個孩子：$B_0, B_1, \dots, B_{k-1}$

Min-max heap

- A complete binary tree
- The root level is min level
- x at min level: x is the smallest key in this subtree.
- x at max level: x is the largest key in this subtree.

insert

at max level

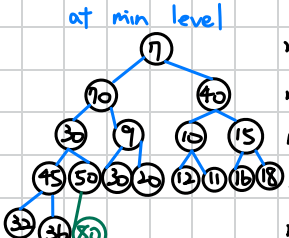


min / max / min / max

1° If insert a node smaller than its parent, then exchange them and keep comparing with the above min levels.

2° Otherwise, keep comparing with the above max levels

at min level



min / max / min / max / min

1° If insert a node larger than its parent, then exchange them and keep comparing with the above max levels.

2° Otherwise, keep comparing with the above min levels

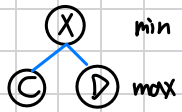
deleteMin

Let the last node X be the root. It has a child C that is the smallest node among X's children and grandchildren

X may not have a child

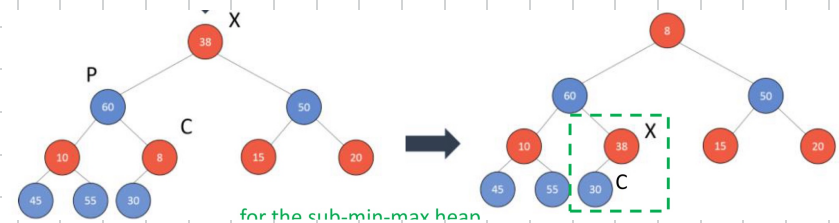
孫辈的 grandchild

① If X.key > C.key and C is a child of the root, then switch locations of X and C.



② If X.key > C.key and C is a grandchild of root,

- Assume P is the parent of C
- Switch locations of X and C
- If X.key > P.key, then switch locations of X and P
- Repeat the process for the subtree rooted at C



for the sub-min-max heap

If j > len, then the corresponding node of i is the counterpart of the parent of i

Deap

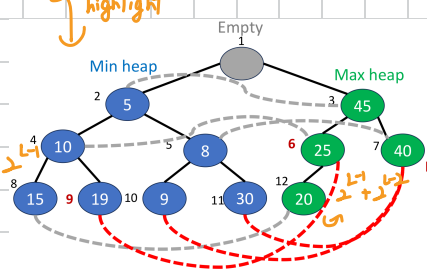
※ Compare with dual(i) whenever the position changes

- A complete binary tree
- The root contains no element
- The left subtree is a min heap
- The right subtree is a max heap

- Let i denote a node in min heap
  Let j denote a corresponding node of i in max heap

$$Level = L = floor(\log_2 i) + 1 = floor(\log_2 j) + 1$$

highlight



insert

- Insert x at the rear and compare with dual(x)
- Maintain the min heap and the max heap

$$j = i + 2^{\lfloor \log_2 i \rfloor - 1}$$

deleteMin

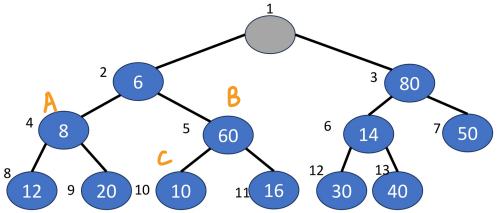
$$j = floor(\frac{j}{2}) \text{ if } j > n$$

- Remove the root of min heap
- Shift the empty node to the leaf position
- Reinsert the rear into the empty position

※ i 對比 dual(i) 小3

∴ min heap 裡不可能有 node 一定小於它 root of max heap

SMMH (Symmetric min-max heap)



A<C<B

- A complete binary tree
- The root contains no element
- Given any node N in the heap
  - The left child of N has the minimum element in elements(N)
  - The right child of N has the maximum element in elements(N)

Insert

- Insert x at the rear
- Compare with its sibling
- Use the relation A<C<B to keep comparing with the above levels

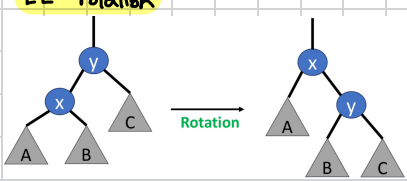
DeleteMin

- Replace the min with the rear
- Compare with its sibling
- Use the relation A<C<B to compare with its leftchild and that of its sibling (the smallest one has higher priority of exchange)

AVL

- Maintain height-balanced
- Preserve the in-order invariant during rotation

LL rotation



- x becomes the new root
- The rightchild of x becomes the leftchild of y
- y becomes the rightchild of x

RR rotation The converse of LL rotation

LR rotation RR rotation + LL rotation

BF (balance factor)

For every node x, BF(x) = height(left subtree of x) - height(right subtree of x)

should be -1, 0, 1.

Hence, the worst case of an AVL tree is $N_h = N_{h-1} + N_{h-2} + 1$ (BF(root) = 1 or -1), which indicates the property of height $O(\log n)$.

log ∵ the number of nodes increases in exponential ∅

∴ the height increase in log(n)

Insert

After insertion, keep checking their BF from x's parent to the root.

- BF = 0 → The subtree becomes a better tree. Stop. → 補到空，的那一邊，原 subtree 高度不變
- BF = ±2 → Process rotation. Stop. → rotate 後，回到原 subtree 高度
- BF = ±1 → Keep moving upward. → subtree 高度 +1
- else → This is not an AVL tree

Delete

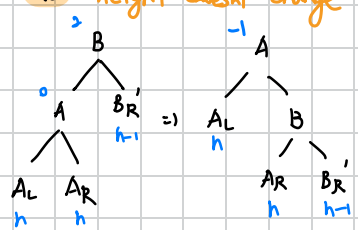
After deletion, keep checking their BF from x's parent to the root.

- BF = ±1 → Stop. → 原 subtree 高度不變
- BF = ±2 → Process rotation. Keep moving upward. → rotate 後，原 subtree 高度降低 1
- BF = 0 → Keep moving upward. → 原 subtree 高度降低 1
- else → This is not an AVL tree

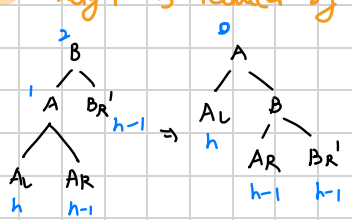
Rotate (while deletion)

Premise ① BF(X) = 2 ② delete right subtree of X ③ Results vary with BF(K)

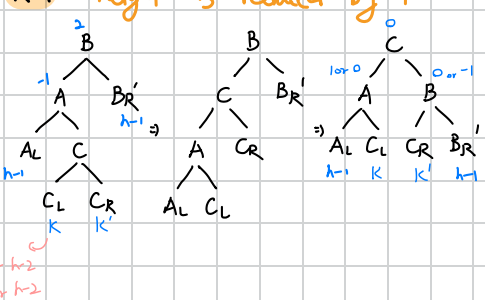
R0 height doesn't change



R1 height is reduced by 1



R-1 height is reduced by 1

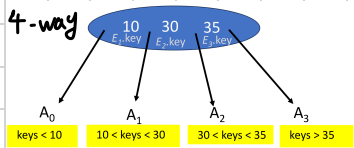


K = h-1 or h-2
K' = h-1 or h-2

M-way search tree

n：該節點存了n筆資料，E_i：窗際的資料，A_i：指向子樹的指標

$n, A_0, E_1, A_1, E_2, A_2, \cdots, E_n, A_n$



- Each node has up to m-1 data pairs and m children
- Key of left data pair < key of right data pair
- E_i.key < all data in the subtree A_i < E_{i+1}.key
- The subtrees are also m-way search trees

Full-degree m tree

- All internal nodes are m-nodes
- # of nodes = $1+m+m^2+\cdots+m^{h-1} = \frac{m^h-1}{m-1}$
- Number of data pairs = $\frac{m^h-1}{m-1}\cdot(m-1) = m^h-1$

B-Tree

※ B-tree of order 2 (m=2) is full binary tree

- The degree of root ≥ 2
- The degree of all internal nodes ≥ $\lceil m/2 \rceil$ → has at least $\lceil m/2 \rceil$ children
- All external node are on the same level

2-3 tree

- m=3 • The internal nodes are either 2-node or 3-node

2-3-4 tree

- m=4 • The internal nodes are 2-node, 3-node, or 4-node

3-4-5 tree

- m=5 • The internal nodes are 3-node, 4-node, or 5-node
- The root may be 2-node

Split an overfull node

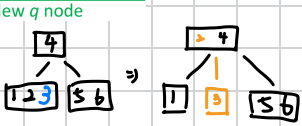
$m, A_0, E_1, A_1, \ldots, E_{\lceil m/2 \rceil -1}, A_{\lceil m/2 \rceil -1}, E_{\lceil m/2 \rceil}, A_{\lceil m/2 \rceil}, E_{\lceil m/2 \rceil +1}, \ldots, E_m, A_m$

p node　　New q node

→ 代表子樹世會被分離之

After insertion, the node p has m data pairs (n=m)

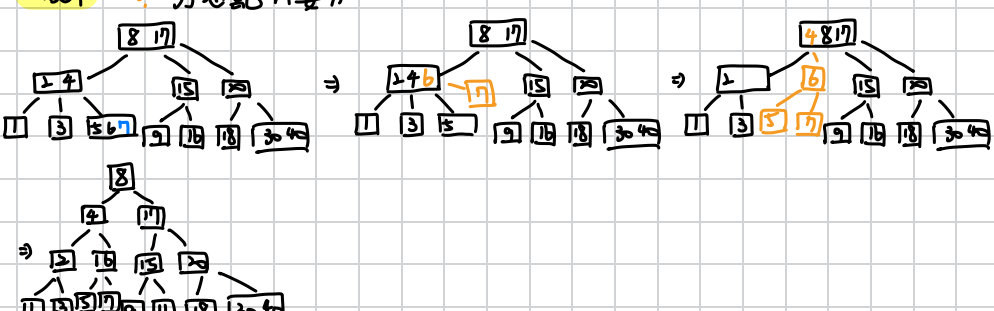
- Split the node p into to two nodes p and q
- Then insert $E_{\lceil m/2 \rceil}$ (a pointer to q) into parent node



Insert

※ 別忘記 n 要加

2-3 tree



Delete

※別忘記 n要減

Delete a data pair x from a leaf node p

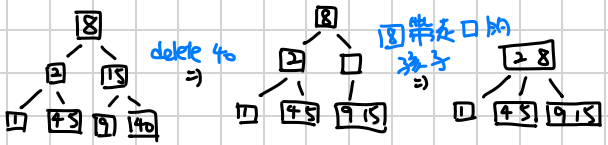
p is the root

- p has at least 2 pairs → Remove x
- p only has 1 pair → Exchange x and the smallest data pair in the right subtree. Remove x.

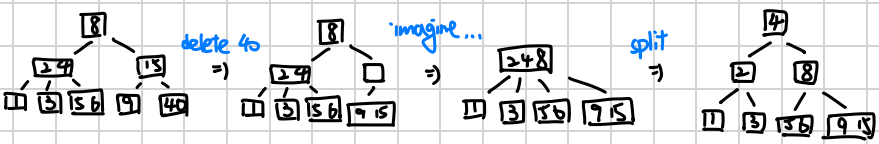
p is not the root (q: the nearest neighbor of p (if any))

- Case 1 → p has at least $\lceil m/2 \rceil$ data pairs ⇒ remove x
- Case 2 → p has $\lceil m/2 \rceil - 1$ data pairs and q has at least $\lceil m/2 \rceil$ data pairs
  ⇒ remove x and do rotation → If unable to do, we do combine
- Case 3 → p has $\lceil m/2 \rceil - 1$ data pairs and q has $\lceil m/2 \rceil - 1$ data pairs
  ⇒ ① remove x and do combine ② check the parent → The height may reduce

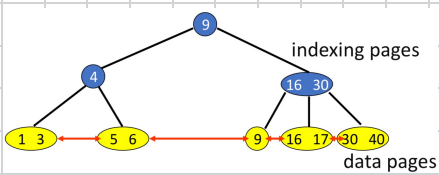
ex①



ex②



B+-Tree



index node

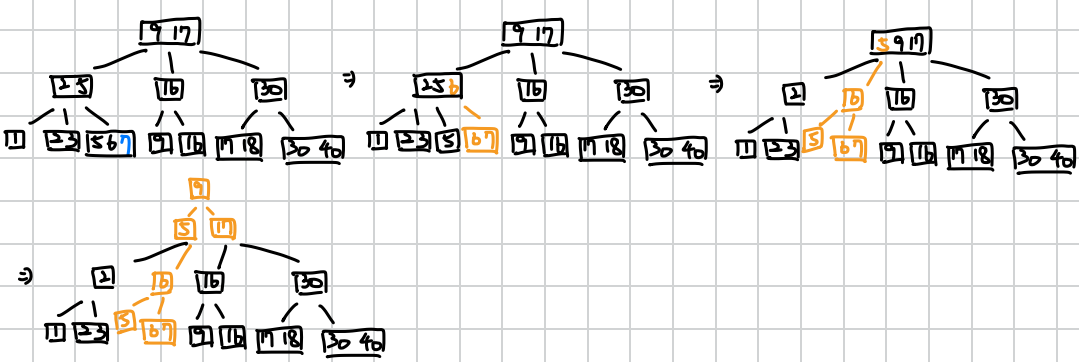
- Internal nodes
- Only store key, not
- Store up to m-1 keys and m pointers
- Left key < Right key
- $K_i \le$ data in the subtree $A_i < K_{i+1}$

data node

- Store data
- There're connections between data
- Store up to m-1 data

data node 的 capacity 的意思

insert



delete

Remove x from the data node p

p has sufficient data → • The key does not need update.

p has insufficient data →
- its sibling has sufficient data → ① • Do rotate. Update the key to the presently smallest key of its sibling node.
- its sibling has insufficient data → ② • Remove the right bound key of x.

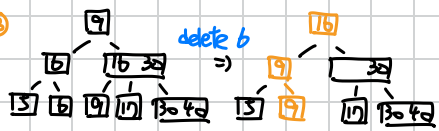
① 

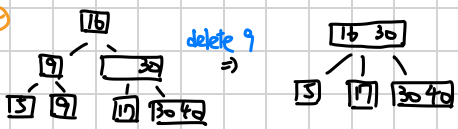
② 

left side has insufficient data → ③ • Remove the corresponding key of x. Get data from its sibling. Do rotate.

right side has sufficient data → ④ • Remove the corresponding key of x. Combine all index nodes with all data nodes.

right side has insufficient data

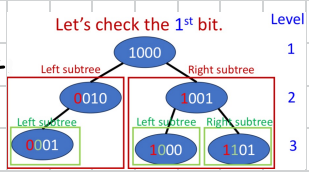
③ 

④ 

Digital search tree

Time complexity: O(# bits in a key)

- Keys are fixed-length
- Number the bit from left to right
- Keys of pairs in its left subtree: the i-th bit = 0
- Keys of pairs in its right subtree: the i-th bit = 1



Search Insert

- Sequentially compare the bits from left to right

Delete

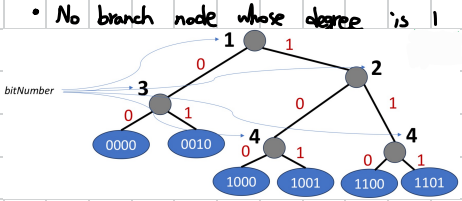
- Remove x • Replace x with any leaf in the subtree rooted at x

只要 key 值是 32-bit int，即使輸入 10^6 筆資料，height is limited to 32 or 33，但還是會有 height-balanced 的問題

Compressed Binary Trie

tells which bit of the key to use to decide the direction to move

- No branch node whose degree is 1 • Add a bitNumber field to each branch node



Delete

- Remove x • Connect x to the grandparent node of x. Remove the parent node of x.

↳ maintain the property of Compressed Binary Trie

Patricia structure of node: bitNumber | leftChild | Data Pair | rightChild

- Additional one node is added as root: header node
- Element nodes point to branch nodes → 1次 compressed binary tree 的特性看

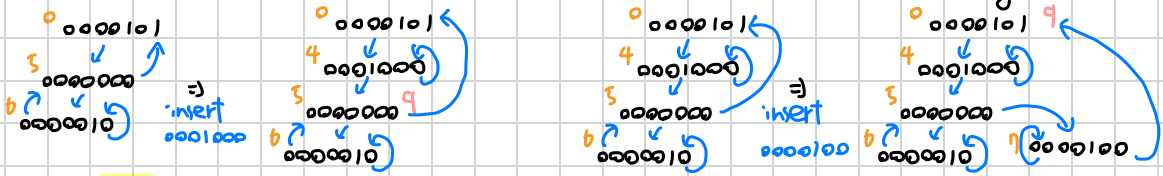
Search

- Go down until bitNumber(now) < bitNumber(previous).
- Check if the present data p equals x

Insert

If x is found →

- Check the first bit that p and x differ
- Add a new node with the bitNumber, x, leftChild, and rightChild



Delete

p has one self pointer →

- Find the parent node of x. Set it as p. → 1次 Compressed Binary Trie 的特性看
  Let p point to another child of p. Remove x.

p has no self pointer →

- Re-search x to find the previous node of x. Set it as q.
  Cover the value of x with that of q. Delete q.